

Numpy

1.pip安装

2.引入numpy

```
import numpy as np
```

1.数组的属性

```
.dtype #数据类型, int,float  
.size #元素总数  
.ndim #维度, []一维, [[]]二维  
.shape #行列数, (行, 列)
```

2.元素填充

```
.zeros((1,2)) #1行2列全0  
.ones((1,2)) #1行2列全1  
.ones((1,2))*2 #1行2列全2  
.empty((1,2)) #1行2列全接近0的数
```

3.简单使用

```
.linspace(0,10,5) #把从0到10的元素分5段
```

```
study-np > 1.py > ...  
1 import numpy as np  
2 a = np.linspace(0,10,5)  
3 print(a)
```

```
● (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts  
python.exe c:/Users/16131/Desktop/VS/study-np/1.py  
[ 0.  2.5  5.  7.5 10. ]  
○ (.venv) PS C:\Users\16131\Desktop\VS> 
```

```
.arange(0,10,2) #从0开始步长为2到10结束, 左闭右开
```

```
study-np > 1.py > ...  
1 import numpy as np  
2 a = np.arange(0,10,2)  
3 print(a)
```

```

• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
python.exe c:/Users/16131/Desktop/VS/study-np/1.py
[0 2 4 6 8]
○ (.venv) PS C:\Users\16131\Desktop\VS>

```

`.arange(5)` #从0开始取到5

```

study-np > 1.py > ...
1 import numpy as np
2 a = np.arange(5)
3 print(a)

```

```

• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
python.exe c:/Users/16131/Desktop/VS/study-np/1.py
[0 1 2 3 4]
○ (.venv) PS C:\Users\16131\Desktop\VS>

```

`.clip(a,5,10)` #将数组a中比5小的元素改为5，比10大的元素改为10，其余不变

```

1 import numpy as np
2 a = np.array([1,2,5,8,10,12])
3 b = np.clip(a,5,10)
4 print(a)
5 print(b)

```

```

• python.exe c:/Users/16131/Desktop/VS/study-np/1.py
[ 1  2  5  8 10 12]
[ 5  5  5  8 10 10]
○ (.venv) PS C:\Users\16131\Desktop\VS>

```

`.nonzero()` #输出非0元素的索引，二维则前行索引对应后列索引

```

study-np > 1.py > ...
1 import numpy as np
2 a = np.nonzero([1,2,5,8,10,12])
3 print(a)

```

```

• python.exe c:/Users/16131/Desktop/VS/study-np/1.py
(array([0, 1, 2, 3, 4, 5]),)
○ (.venv) PS C:\Users\16131\Desktop\VS>

```

```

study-np > 1.py > ...
1 import numpy as np
2 a = np.nonzero([[1,2,3]])
3 print(a)

```

```
• \python.exe c:/Users/16131/Desktop/VS/study-np/1.py  
(array([0, 0, 0]), array([0, 1, 2]))  
o (.venv) PS C:\Users\16131\Desktop\VS> |
```

```
1 import numpy as np  
2  
3 a = np.nonzero([[0, 1, 0],  
4               [2, 0, 3],  
5               [0, 4, 0]])  
6  
7 print (a)
```

```
• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts  
\python.exe c:/Users/16131/Desktop/VS/study-np/1.py  
(array([0, 1, 1, 2]), array([1, 0, 2, 1]))  
o (.venv) PS C:\Users\16131\Desktop\VS> |
```

4.简单运算

- 能求总的, +aixs=1/0, 求逐行或列

```
.max() #找最大值  
.min() #找最小值
```

```
study-np > 1.py > ...  
1 import numpy as np  
2  
3 a = np.max([[0, 1, 0],  
4           [2, 0, 3],  
5           [0, 4, 0]])  
6  
7 print (a)
```

```
• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts  
\python.exe c:/Users/16131/Desktop/VS/study-np/1.py  
4
```

```
.argmax()或.argmaxmin() +aixs=1/0 #逐行或逐列找最大或最小, 返回索引, 若有俩只返回第一个的索引
```

```
study-np > 1.py > ...
1 import numpy as np
2
3 a = np.argmax([[0, 1, 2],
4               [2, 3, 3],
5               [0, 4, 5]],axis = 1)
6
7 b = np.argmin([[0, 1, 2],
8               [2, 3, 3],
9               [0, 4, 5]],axis = 0)
10 print (a)
11 print (b)
```

```
(.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
\python.exe c:/Users/16131/Desktop/VS/study-np/1.py
• [2 1 2]
  [0 0 0]
```

.sum #求总和

```
study-np > 1.py > ...
1 import numpy as np
2
3 a = np.sum([[0, 1, 0],
4             [2, 0, 3],
5             [0, 4, 0]])
6
7 print (a)
```

```
• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
\python.exe c:/Users/16131/Desktop/VS/study-np/1.py
10
○ (.venv) PS C:\Users\16131\Desktop\VS>
```

.sum(axis=1) #逐行求和
.sum(axis=0) #逐列求和

```

1  import numpy as np
2
3  a = np.sum([[0, 1, 2],
4             [2, 3, 3],
5             [0, 4, 5]],axis = 1)
6
7  b = np.sum([[0, 1, 2],
8             [2, 3, 3],
9             [0, 4, 5]],axis = 0)
10 print (a)
11 print (b)

```

```

(.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
python.exe c:/Users/16131/Desktop/VS/study-np/1.py
[3 8 9]
[ 2  8 10]

```

.median() #求中位数

```

study-np > 1.py > ...
1  import numpy as np
2
3  a = np.median([[0, 1, 2],
4                [2, 3, 3],
5                [0, 4, 5]])
6  print (a)

```

```

(.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
python.exe c:/Users/16131/Desktop/VS/study-np/1.py
2.0

```

.mean() 或者 .average() #求平均值

```

study-np > 1.py > ...
1  import numpy as np
2  ✨
3  a = np.mean([[0, 1, 2],
4               [2, 3, 3],
5               [0, 4, 5]])
6  print (a)
7

```

```

(.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
python.exe c:/Users/16131/Desktop/VS/study-np/1.py
2.2222222222222223

```

.sort() #逐行排序

```
study-np > 1.py > ...
1  import numpy as np
2
3  a = np.sort([[0, 1, 0],
4              [2, 0, 3],
5              [0, 4, 0]])
6
7  print (a)
```

```
• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
\python.exe c:/Users/16131/Desktop/VS/study-np/1.py
[[0 0 1]
 [0 2 3]
 [0 0 4]]
```

+, -, *, ** #正常按位对应运算

```
1  import numpy as np
2
3  a = np.array([[0, 1, 0],
4              [2, 0, 3],
5              [0, 4, 0]])
6
7  b = np.array([[0, 1, 0],
8              [2, 0, 3],
9              [0, 4, 0]])
10 print (a+b)
11 print(a*b)
12 print(a-b)
13 print(a**2)
```

```
• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
\python.exe c:/Users/16131/Desktop/VS/study-np/1.py
[[0 2 0]
 [4 0 6]
 [0 8 0]]
[[ 0  1  0]
 [ 4  0  9]
 [ 0 16  0]]
[[0 0 0]
 [0 0 0]
 [0 0 0]]
[[ 0  1  0]
 [ 4  0  9]
 [ 0 16  0]]
```

`.transpose()` #求转置T

```
study-np > 1.py > ...
1  import numpy as np
2
3  a = np.transpose([[0, 1, 0],
4                  [2, 0, 3],
5                  [0, 4, 0]])
6
7  print (a)
```

```
(.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
python.exe c:/Users/16131/Desktop/VS/study-np/1.py
• [[0 2 0]
   [1 0 4]
   [0 3 0]]
```

`.dot(a,b)` #矩阵乘法

```
study-np > 1.py > ...
1  import numpy as np
2
3  a = np.array([[0, 1, 0],
4               [2, 0, 3],
5               [0, 4, 0]])
6
7  b = np.array([[0, 1, 0],
8               [2, 0, 3],
9               [0, 4, 0]])
10 print (np.dot(a,b))
```

```
(.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
python.exe c:/Users/16131/Desktop/VS/study-np/1.py
• [[ 2  0  3]
   [ 0 14  0]
   [ 8  0 12]]
(.venv) PS C:\Users\16131\Desktop\VS>
```

`.cumsum()` #递增位数求和

```

1  import numpy as np
2
3  a = np.cumsum([[0, 1, 2],
4                [2, 3, 3],
5                [0, 4, 5]])
6
7
8  print (a)

```

```

• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts\python.exe c:/Users/16131/Desktop/VS/study-np/1.py
[ 0  1  3  5  8 11 11 15 20]

```

```

1  import numpy as np
2
3  a = np.cumsum([[0, 1, 2],
4                [2, 3, 3],
5                [0, 4, 5]],axis = 1)
6
7
8  print (a)

```

```

• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts\python.exe c:/Users/16131/Desktop/VS/study-np/1.py
[[0 1 3]
 [2 5 8]
 [0 4 9]]

```

.diff() #求相邻数的差

```

study-np > 1.py > ...
1  import numpy as np
2
3  a = np.diff([[0, 1, 2],
4              [2, 3, 3],
5              [0, 4, 5]])
6
7
8  print (a)

```

```

• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
\python.exe c:/Users/16131/Desktop/VS/study-np/1.py
[[1 1]
 [1 0]
 [4 1]]
• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
\python.exe c:/Users/16131/Desktop/VS/study-np/1.py
Traceback (most recent call last):
  File "c:\Users\16131\Desktop\VS\study-np\1.py", line 3, in <module>
    a = np.diff([[0, 1, 2],
                ^^^^^^^^^^^^^^^^^
TypeError: diff() got an unexpected keyword argument 'axis'
• (.venv) PS C:\Users\16131\Desktop\VS>

```

5.索引、分割

A[] #单行位置索引
 A[] [] 或 A[,] #多行多列位置索引，前行后列
 A[:,n] #所有用：表示，所有行的n列
 A[m,:] #所有用：表示，m行的所有列
 A[m:n,f] #从第m行到n行，左闭右开，取第f列的元素
 A[f,m:n] #取f行元素，从第m列到n列，左闭右开

```

study-np > 1.py > ...
1  import numpy as np
2
3  a = np.array([[0, 1, 2],
4               [3, 4, 5],
5               [6, 7, 8],
6               [9, 14, 15]])
7
8
9  print (a[1:3,1])

```

```

• (.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
\python.exe c:/Users/16131/Desktop/VS/study-np/1.py
[4 7]

```

for row in A: 或 for column in A.T #逐行输出
 for rom in A.T: #逐列输出
 for item in a.flat: #逐项输出（换行）
 a.flatten() #逐项输出，放一行

```

1  import numpy as np
2
3  a = np.array([[0, 1, 2],
4               [3, 4, 5],
5               [6, 7, 8],
6               [9, 14, 15]])
7
8  for item in a.flat:
9      print(item)
10
11 print (a.flatten())
12

```

```

(.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
\python.exe c:/Users/16131/Desktop/VS/study-np/1.py
0
1
2
3
4
5
6
7
8
9
14
15
[ 0  1  2  3  4  5  6  7  8  9 14 15]

```

.vstack((A,B))或.concatenate((A,B),axis=0)#纵向合并矩阵A,B
 .hstack((A,B))或.concatenate((A,B),axis=1)#横向合并矩阵A,B
 .newaxis #一维数组用不了转置，这里相当于一维数组的转置

```

study-np / 1.py / ...
1  import numpy as np
2
3  A = np.array([0, 1, 2])
4
5  print (A[:,np.newaxis])

```

```

(.venv) PS C:\Users\16131\Desktop\VS> & c:\Users\16131\Desktop\VS\.venv\Scripts
\python.exe c:/Users/16131/Desktop/VS/study-np/1.py
[[0]
 [1]
 [2]]

```

```
.vsplit(A,m) #纵向分割m块  
.hsplit(A,m) #横向分割m块  
.split(A,m,axis=1) #将A横向（横的切）等个数分割成3部分  
.split(A,m,axis=0) #将A纵向（竖的切）等个数分割成3部分
```

用索引改变某项，所有相关都变

```
b=a.copy() #修改a后b不更改
```